

# 最优化技术

孙志强

## 第五章 有约束非线性优化问题

# 内容

## 1 5.8 罚函数法

## 2 总结

# 引言

已经学习的内容：

- ① 二次规划
  - ▶ 起作用约束集法
- ② 线性约束下的非线性规划(非二次规划)
  - ▶ Zoutendijk方法
  - ▶ 梯度投影法

# 引言

已经学习的内容：

- ① 二次规划
  - ▶ 起作用约束集法
- ② 线性约束下的非线性规划(非二次规划)
  - ▶ Zoutendijk方法
  - ▶ 梯度投影法

本次课的学习内容

- ① 一般化的非线性优化问题求解方法

## 求解方法

着重学习以下两大类方法

- 1 罚函数法
- 2 逐步二次规划(SQP)法

这也是MATLAB优化工具箱中所使用的方法，应用比较广泛，尤其是SQP方法。

# 优化问题模型

$$(P) \begin{cases} f(x) \rightarrow \min \\ g_i(x) \leq 0 & i \in \mathcal{I} = \{1, 2, \dots, m\} \\ h_j(x) = 0 & j \in \mathcal{E} = \{1, 2, \dots, p\} \end{cases}$$

$\mathcal{F}$ 表示可行域.

## 基本思路

将复杂的问题转化为简单的问题！什么问题简单？

## 基本思路

将复杂的问题转化为简单的问题！什么问题简单？  
无约束问题！



## 基本思路

将复杂的问题转化为简单的问题！什么问题简单？

无约束问题！

可以尝试将有约束的问题转换为无约束的问题进行求解。这是罚函数法的基本思路。常见的做法是在目标函数上增加一个罚项。罚项的作用是当迭代点不在可行域中时，尝试将其拉进可行域；或迭代点试图脱离可行域时，阻止其离开。这是罚函数法两种不同的思路，由此分别得到了外点罚函数法和内点罚函数法。

## 问题描述

一个有约束非线性优化问题

$$(P) \begin{cases} f(x) \rightarrow \min \\ x \in \mathcal{F} \end{cases}$$

## 示性函数及其特性

$$\delta_{\mathcal{F}}(x) = \begin{cases} 0, & x \in \mathcal{F} \\ \infty, & x \notin \mathcal{F}. \end{cases}$$

将 $\delta_{\mathcal{F}}(x)$ 加到目标函数上, 可得 $F(x) = \delta_{\mathcal{F}}(x) + f(x)$ 。  
如果对函数 $F(x)$ 求最小化, 会发现什么问题?

## 示性函数及其特性

$$\delta_{\mathcal{F}}(x) = \begin{cases} 0, & x \in \mathcal{F} \\ \infty, & x \notin \mathcal{F}. \end{cases}$$

将 $\delta_{\mathcal{F}}(x)$ 加到目标函数上, 可得 $F(x) = \delta_{\mathcal{F}}(x) + f(x)$ 。

如果对函数 $F(x)$ 求最小化, 会发现什么问题?

当 $x \notin \mathcal{F}$ 时,  $F(x) \rightarrow \infty$ 。

因此,  $F(x)$ 要想获得最小值, 必须在原问题的可行域 $\mathcal{F}$ 中取值。

## 问题转换

这就意味着求解  $F(x) \rightarrow \min$  就等价于求解

$$(P) \begin{cases} f(x) \rightarrow \min \\ x \in \mathcal{F} \end{cases}$$

这样一来，就将有约束的问题转换为了无约束的问题。示性函数  $\delta_{\mathcal{F}}(x)$  即为罚项。

## 选择罚项

无约束非线性优化问题的求解方法中，除了单纯形替换法之外，都需要计算目标函数的梯度，甚至是黑塞矩阵。这意味着目标函数应该连续，至少一阶可微。但是，目标函数

$$F(x) = f(x) + \delta_{\mathcal{F}}(x)$$

无法做到这一点，因为示性函数 $\delta_{\mathcal{F}}(x)$ 不是连续的。因此，在选择罚项时，应该使得罚项函数足够平滑。一种较好的做法是选择一个足够平滑的函数 $\pi(x)$ ，当 $x \in \mathcal{F}$ 时， $\pi(x) = 0$ ；当 $x \notin \mathcal{F}$ 时， $\pi(x) > 0$ ；且满足

$$\frac{1}{r}\pi(x) \rightarrow \delta_{\mathcal{F}}(x) \quad \text{for} \quad 0 < r \rightarrow 0$$

$r$ 称为罚因子。

## 选择罚项

可行域:

$$\mathcal{F} = \{x \in \mathbb{R}^n \mid g_i(x) \leq 0, i \in \mathcal{I}; h_j(x) = 0, j \in \mathcal{E}\}$$

罚项:

$$\pi(x) = \sum_{i=1}^m [g_i^+(x)]^\alpha + \sum_{j=1}^p |h_j(x)|^\alpha$$

选择  $\alpha \geq 1$  (比如,  $\alpha = 2$ ),  $a^+ = \max\{0, a\}$ ,  $a$  为实数,  
 $g^+(x) = (g(x))^+$ .

# 示例

$p = 0, m = 2$ , 约束项为

$$g_1(x) = -1 - x, g_2(x) = x - 1$$

选择  $\alpha = 2$ , 罚项  $\pi(x)$ :

$$\pi(x) = \begin{cases} (x+1)^2 & x \leq -1 \\ 0 & -1 \leq x \leq 1 \\ (x-1)^2 & x \geq 1 \end{cases}$$

选择  $\alpha = 1$ , 罚项  $\pi(x)$ :

$$\pi(x) = \begin{cases} -x-1 & x \leq -1 \\ 0 & -1 \leq x \leq 1 \\ x-1 & x \geq 1 \end{cases}$$



## 外点法的算法

### 算法步骤

- 1 选择初始点 $x^{(0)}$ ，构造罚因子序列 $r_k$ ，给出终止条件的精度 $\epsilon > 0$ ，令 $k = 1$ 。
- 2 构造增广目标函数 $F(x) = f(x) + \frac{1}{r}\pi(x)$ 。
- 3 选择一种合适的方法求解无约束的非线性规划

$$F(x) \rightarrow \min$$

罚因子序列应该逐渐趋向于0，如可按照 $r_k = 1/k$ 的方式进行选择。

# 示例

$$\begin{aligned} f(x) &= x_1^2 + 4x_1x_2 + 5x_2^2 - 10x_1 - 20x_2 \rightarrow \min \\ h_1(x) &= x_1 + x_2 - 2 = 0 \end{aligned} \quad (1)$$

罚项为  $\pi(x) = [h_1(x)]^2$ ,

$$F(x) = f(x) + \frac{1}{r}[h_1(x)]^2$$

## 示例

$F(x)$ 的梯度为

$$\nabla F(x) = \begin{pmatrix} 2x_1 + 4x_2 - 10 + \frac{2}{r}(x_1 + x_2 - 2) \\ 4x_1 + 10x_2 - 20 + \frac{2}{r}(x_1 + x_2 - 2) \end{pmatrix}$$

求解 $\nabla F(x) = 0$ ,

$$x_1^*(r) = \frac{5r + 1}{r + 2}; x_2^*(r) = \frac{3}{r + 2}$$

当 $r \rightarrow 0$ 时,  $x_1^*(r) \rightarrow [0.5, 1.5]^T$

## 示例

$F(x)$ 的黑塞矩阵为

$$\nabla^2 F(x) = \begin{pmatrix} 2 + \frac{2}{r} & 4 + \frac{2}{r} \\ 4 + \frac{2}{r} & 10 + \frac{2}{r} \end{pmatrix}$$

黑塞矩阵的条件数 $\kappa \approx \frac{2}{r}$ . 当 $r$ 取较小值时,黑塞矩阵是病态的.条件数是最大特征值与最小特征值之间的比值.

## 外点法的特性

- 将有约束的问题转换为比较简单的无约束问题进行求解.
- 罚项不断减小, 有可能导致目标函数的黑塞矩阵出现病态, 导致数值算法不可用.
- 初始搜索点可以不是可行点.

## 基本思路

与外点法方法类似，障碍函数法，又称为内点罚函数法(注意同内点法的区别)也是通过增加一个罚项，将有约束问题转换为无约束问题进行求解。与外点罚函数法不同，障碍函数法是将迭代点完全约束在可行域内进行，对试图离开可行域的点进行惩罚，从而达到将它们约束在可行域中的目的。

## 基本结构

在障碍函数法中，增广目标函数为

$$F(x) = f(x) + rB(x)$$

$B(x)$ 为罚项，也称为障碍函数。 $B(x)$ 应该具有什么样的特性，才能够将迭代点始终约束在可行域内？

## 基本结构

在障碍函数法中，增广目标函数为

$$F(x) = f(x) + rB(x)$$

$B(x)$ 为罚项，也称为障碍函数。 $B(x)$ 应该具有什么样的特性，才能够将迭代点始终约束在可行域内？当迭代点试图离开可行域中， $B(x)$ 的值应该趋向于 $\infty$ ；而当迭代点是可行域的严格内点时， $B(x)$ 的值应该为零。即

$$B(x) = \begin{cases} 0 & x \in \overset{\circ}{\mathcal{F}} \\ \infty & x \in \partial\mathcal{F} \end{cases}$$

$\overset{\circ}{\mathcal{F}}$ 表示可行域的内部， $\partial\mathcal{F}$ 表示可行域的边界。



## 障碍函数

### 倒数型障碍函数

$$B(x) = - \sum_{i=1}^m \frac{1}{g_i(x)}$$

### 对数型障碍函数

$$B(x) = - \sum_{i=1}^m \log(-g_i(x))$$

和外点罚函数法类似，障碍函数法也引入了罚因子 $r$ ，使得增广目标函数更为平滑：

$$F(x) = f(x) + rB(x)$$

## 示例

$$f(x) = x_1 + x_2 \rightarrow \min$$

$$g_1(x) = x_1^2 - x_2 \leq 0$$

$$g_2(x) = -x_1 \leq 0$$

选择对数型障碍函数

$$F(x) = x_1 + x_2 - r \log(-x_1^2 + x_2) - r \log(x_1)$$

# 示例

$F(x)$ 的梯度为

$$\nabla F(x) = \begin{pmatrix} 1 + \frac{2rx_1}{-x_1^2+x_2} - \frac{r}{x_1} \\ 1 - \frac{r}{-x_1^2+x_2} \end{pmatrix}$$

令 $\nabla F(x) = 0$

$$x(r) = \begin{pmatrix} x_1(r) \\ r + x_1(r)^2 \end{pmatrix}$$

$x_1(r) = \frac{1}{4}(-1 + \sqrt{1 + 8r})$ . 随着 $r \rightarrow 0$ ,  $x^* = (0, 0)^T$ .

## 模型

在Matlab中，内点罚函数法（障碍函数法）的定义为：对于优化问题

$$\min f(x), \text{ subject to } h(x) = 0 \quad \text{and} \quad g(x) \leq 0$$

按照如下方式构造增广目标函数

$$\begin{cases} \min f(x, s) = \min[f(x) - \mu \sum_i \log(s_i)] \\ \text{subject to } h(x) = 0, g(x) + s = 0 \end{cases}$$

$s$ 为松弛变量。

## 函数fmincon

Matlab定义的用于求解有约束非线性规划的函数为fmincon，所对应的数学模型为

$$\begin{cases} f(x) \rightarrow \min \\ c(x) \leq 0 \\ ceq(x) = 0 \\ A \cdot x \leq b \\ Aeq \cdot x = beq \\ lb \leq x \leq ub \end{cases}$$

$c(x)$ 和 $ceq(x)$ 分别表示不等式和等式非线性约束方程； $A$ 和 $Aeq$ 则表示不等式和等式线性约束方向的系数矩阵。

## 函数fmincon

fmincon的调用方式:

```
x = fmincon(fun,x0,A,b)
x = fmincon(fun,x0,A,b,Aeq,beq)
x = fmincon(fun,x0,A,b,Aeq,beq,lb,ub)
x = fmincon(fun,x0,A,b,Aeq,beq,lb,ub,nonlcon)
x = fmincon(fun,x0,A,b,Aeq,beq,lb,ub,nonlcon,...
            options)
```

与linprog和quadprog不同，fmincon增加了一个非线性约束项，用参数nonlcon表示。目标函数也是非线性的，因此，也无法采用向量或矩阵的形式进行表示。

## 目标函数和非线性约束的定义方式

非线性约束采用函数(function)的方式进行定义:

```
function [c,ceq] = mycon(x)
c = ...    % Compute nonlinear inequalities at x.
ceq = ...  % Compute nonlinear equalities at x.
```

类似的, 目标函数也采用函数的方式进行定义:

```
function f = objfun(x)
f = ...           % Compute function value at x
```

调用格式为

```
x = fmincon(@objfun,x0,A,b,Aeq,beq,lb,ub,@mycon)
```

## fmincon的输出

```
[x,fval] = fmincon(...)  
[x,fval,exitflag] = fmincon(...)  
[x,fval,exitflag,output] = fmincon(...)  
[x,fval,exitflag,output,lambda] = fmincon(...)  
[x,fval,exitflag,output,lambda,grad] = fmincon(...)  
[x,fval,exitflag,output,lambda,grad,hessian] = ...  
fmincon(...)
```

grad表示目标函数的梯度，hessian表示目标函数的Hessian矩阵。其余的输出参数与linprog和quadprog相同。



## 示例

针对前面的示例利用Matlab进行求解，代码为：

%定义约束方程

```
function [c,ceq]=mycon(x)
c=[x(1)^2-x(2)      -x(1)];
ceq=[];
```

%定义目标函数

```
function f=objfun(x)
f=x(1)+x(2);
```

%调用fmincon进行求解

```
x0=[0;0];
ops=optimset('Algorithm','interior-point');
[x,fval,exitflag,output,lambda,grad] = fmincon(
@objfun,x0,[],[],[],[],[],[],@mycon,ops)
```

## 示例

计算结果：

最优解 $x = (0, 0)^T$ ，目标函数梯度 $(1, 1)^T$ .

两个约束方程在最优解时对应的Lagrange乘子：

$$\lambda_1 = 1; \lambda_2 = 1$$

# 总结

- ❶ 罚函数法的基本思路
- ❷ 两种罚函数法
  - ▶ 外点罚函数法
  - ▶ 内点罚函数法(障碍函数法)
- ❸ Matlab优化工具箱中的罚函数法